Egyptian Informatics Journal 31 (2025) 100757

Contents lists available at ScienceDirect

Egyptian Informatics Journal

journal homepage: www.sciencedirect.com

Full Length Article

Task scheduling in fog computing systems using deep learning

ZhongYI Huang

Nanjing Institute of Technology, Data Science and Big Data Technology, Nanjing 210000, China

ARTICLE INFO

Keywords:
Task scheduling
Fog computing
Response time
Turnaround time
Waiting time

ABSTRACT

Optimal resource allocation is one of the fundamental challenges in fog computing systems. In this paper, a novel deep learning method is proposed for efficient task allocation and scheduling in fog computing systems. The proposed method consists of a three-phase structure. In the first phase, a stable communication structure between network components is established using a minimum spanning tree structure based on the distance between nodes. In the second phase, a deep neural network is used to determine the appropriate computational resources for each task, in which task characteristics and processing resources are analysed as inputs and the priority of assigning the task to each resource is determined. And finally, in the third phase, after assigning each task to resources, the order and processing time of the tasks are determined separately using an improved version of the round-robin algorithm, and this is done based on the dependencies between tasks. To check our method, we performed experiments by changing the number of running tasks and by altering the average time needed to complete each task. The method is measured against well-known algorithms such as enhanced round-robin, particle swarm optimization and deep reinforcement learning for multi-objective scheduling. The experiments conducted and the values obtained in terms of response time, turnaround time and waiting time show that the proposed method has achieved 9.96 %, 7.80 % and 8.69 % improvements in the first scenario, respectively. Moreover, in the second scenario, these successes have increased to 20.25 %, 8.94 %, and 7.33 %, respectively. These results represent the high efficiency of the proposed method for the optimization of different times and performance improvement of a system under various conditions.

1. Introduction

With the development in digital technologies, a large number of data sets are being produced from different origins [1]. Such information could be preserved as well as accessed with the assistance of cloud computing solutions. The cloud computing is unable to assist the requirements needed for IoT both in terms of mobility as well as security of devices [2]. Thus in 2012, Cisco propounded the fog computing [3]. Fog computing continues the cloud services to the network edge. This reduces time and volume for data transfer due to the execution of related operations on the local resources that are available at the vicinity of IoT edge devices. Consequently, using local resources saves costs, latency, a higher confidential and security level is achieved, as well as a reduced traffic load over the network. In the case of the absence of effective resources in fog computing, cloud resources are used with more cost [4].

By putting cloud computing services closer to the end user, fog computing makes it easier for delay-sensitive applications—like those used in online banking, robotic control of systems, and driverless cars—to function [5,6]. These applications have specific processing time frames, and the output's validity depends on when it becomes available. Failure to satisfy deadlines in a real-time application might result in outcomes ranging from ineffective results to catastrophic occurrences [7].

Several problems in cloud-fog networks and IoT are really optimization challenges [8]. This relates to the scheduling problem at work. An application's tasks must be distributed among the processing components across the Cloud to Things (C2T) continuum once it reaches the edge of the network. Allocating an application's actions to accessible resources in order to achieve predetermined goals is the responsibility of a scheduler [9–11]. Intelligent methods based on machine learning have great potential to solve this problem, and this study has attempted to exploit this potential to provide a new method. Several research opportunities are what motivate this study. At present, the use of machine learning and deep learning (DL) in task scheduling for fog computing is increasing, but most existing methods only address certain parts of the problem individually. The study points out that there is a lack of holistic, step-by-step frameworks that use different techniques to manage the entire scheduling challenge – from setting up network communication to

E-mail address: x00202230421@njit.edu.cn.

https://doi.org/10.1016/j.eij.2025.100757
Received 24 March 2025; Received in revised form 3 June 2025; Accepted 10 August 2025
Available online 2 September 2025

arranging tasks. We hope to solve this by proposing a three-phase method that centers on deep learning. Second, DL has been used to model QoS needs, but it could be improved to directly affect the way resources are allocated. We use a Deep Convolutional Neural Network (DCNN) that is set up to analyze both tasks and resources in detail to support better QoS-aware resource matching. Although inter-task dependencies are recognized, they have not been studied in detail with the use of comprehensive deep learning-based scheduling. It achieves this by adding a step that considers dependencies when scheduling tasks which helps improve system performance.

The research innovations include three main ones, which can be described based on the identified research gaps and the efforts made to address them. The first innovation is the use of CNN for resource allocation. The use of this type of network allows the model to effectively model complex features related to the resource performance pattern. The second innovation investigates the dependencies that exist between the tasks while doing scheduling; such a model makes decisions intelligently about the dynamic order of task execution. At last, this is the research innovation that shows comprehensive system modeling, fully representing various aspects in the fog computing system, with different layers, resources, and users. The contributions of the paper are as follows:

- Creating a new three-phase framework for managing task scheduling in fog computing. The framework does this by: Creating a stable network infrastructure with a Maximum Spanning Tree (based on link quality); Using a CNN to intelligently match tasks to the best fog resources; and employing an improved Round-Robin algorithm that takes dependencies into account for scheduling tasks.
- Showing that a CNN can handle the challenges of assigning tasks in fog networks that are always changing. The CNN can manage: Workloads that change over time and have different quality of service needs; and the efficient allocation of tasks to the different types of fog resources.
- Improving the use of fog computing resources by combining different approaches. The integrated framework clearly improves how resources are used by guaranteeing reliable data transfer, selecting the best resources with CNN and organizing tasks to avoid bottlenecks.

This paper is organized as follows: Research related to the current work is examined in Section 2. Section 3 describes the suggested technique in detail, while Section 4 displays the study's findings. Section 5 presents the conclusions.

2. Related work

Managing tasks and resources well is essential in fog computing to benefit from its low latency and efficient way of processing IoT data. Even so, it is very difficult to design the best scheduling strategies for this sector. These factors are the heterogeneous nature of fog resources (different computing, storage and memory), the changing demands on workloads and networks, the need to meet several QoS requirements (such as strict deadlines, saving energy and reducing the overall response time) and the difficulty in handling dependencies between different tasks in distributed applications. The literature suggests a range of ways to address these complicated topics. They can be classified into the following main groups:

I. Heuristic and Rule-Based Algorithms: which use predefined rules or quick tricks to decide which job to run next (such as Earliest Deadline First, Round-Robin or shortest job first).
II. Metaheuristic and Evolutionary Algorithms: which use methods that are inspired by nature (e.g., Simulated Annealing, Genetic Algorithms, Particle Swarm Optimization, Grey Wolf Optimizer) to find good solutions for complex optimization problems.
III. Machine Learning (ML) and Artificial Intelligence (AI) based approaches, including Deep Learning (DL) and Reinforcement Learning (RL), which leverage data-driven models to learn optimal policies, predict resource needs, or classify tasks for intelligent placement and scheduling.

The following review delves into specific contributions within these categories, highlighting diverse methodologies aimed at enhancing performance in fog environments. These papers analyse various algorithms and methods designed to optimize efficiency and performance in fog computational environment. Najafizadeh et al. [12] presented an effective Multi-Objective Simulated Annealing (MOSA) for task allocation of tasks to fog and cloud nodes securely, together with a goal programming approach to obtain a compromise solution that optimizes multiple objectives. This algorithm outperformed other algorithms while maintaining service costs within acceptable limits.

In order to achieve the best results in terms of waiting time and delay, Hoseini et al. [13] suggested a multi-criterion scheduling technique for fifth-generation mobile fog technology that takes into account completion time, consumption of energy, RAM, and deadline constraints. Yadav et al. [14] presented a hybrid heuristic and metaheuristic method, termed BH-FWA, for job scheduling in fog computing. This methodology integrated the Fireworks Algorithm (FWA) with Heterogeneous Earliest Finish Time (HEFT) to reduce makespan and expenses. Its efficacy was corroborated using parameters including makespan, cost, and throughput.

Navaneetha Krishnan and Thiyagarajan [15] explained the IoT data administration mechanism efficiently with fog computing, targeting task scheduling as the main aim to reduce energy consumptions as well as makespan. They used a heuristic method combined with dynamic voltage frequency scaling, as was obtained by simulations. Azizi et al. [16] introduced two semi-greedy algorithms, termed Priority-aware Semi-Greedy (PSG) and PSG-M, for the effective allocation of IoT tasks to Fog Nodes (FNs). These algorithms enhanced job deadline adherence by 1.35 times and reduced overall violation time by 97.6 %.

Ghafari and Mansouri [17] proposed a novel version of the Artificial Hummingbird Algorithm that optimized chaotic maps and initial populations, named as CODA for fog computing systems. This significantly reduced energy consumption, duration, and costs in contrast with previous algorithms. Saif et al. [18] suggested a multi-objectives approach based on Grey Wolf Optimizer (MGWO) which aims minimizing latency and energy usage associated with Quality of Service (QoS) objectives in cloud-fog computing, evaluating its efficacy against contemporary techniques.

Alhaidari and Balharith [19] proposed the Dynamic Round-Robin Heuristic Algorithm (DRRHA), an innovative task scheduling method in cloud computing systems that markedly enhanced QoS and system sustainability through the dynamic adjustment of the time quantum. The Longest Job to Fastest Processor (LJFP) and Minimum Completion Time (MCT) algorithms were used in an enhanced initialization of Particle Swarm Optimization (PSO) by Alsaidy et al. [20], who evaluated the techniques' effectiveness in lowering makespan, total execution time, imbalance, and energy consumption. Iftikhar et al. [21] proposed HunterPlus, which is a CNN-based resource scheduling approach. They outperformed the Gated Graph Convolution Network (GGCN) and Bidirectional GGCN schedulers for energy consumption and job completion ratio metrics, respectively. This work has underlined that the characteristics of the dataset and the presence of the Bidirectional layer are crucial to improving the performance of schedulers.

Abdel-Basset et al. [22] have introduced the multi-objective modified marine predators algorithm, a task scheduling approach that enhances the performance of the modified marine predators algorithm through the inclusion of polynomial crossover operators and adaptive CF parameters. Choppara and Mangalampalli [23] proposed RTATSA2C, a cloud computing task scheduler that utilized reinforcement learning and the Advantage Actor-Critic model for optimization of scheduling,

improving the reliability and reducing makespan.

In contrast to the current fog schedulers, such as RACE (CFP) and RACE (FOP), Hussain et al. [24] introduced a task scheduling method for IoT called Resource Aware Prioritized Task Scheduling (RAPTS), which enhanced resource consumption, reaction time, and deadline completion. In order to reduce makespan and increase dependability, Hosseini et al. [25] suggested fog processing scheduling for Bag of Task (BoT) Internet of Things applications. It extended a multi-objective discrete cuckoo search algorithm, which provides significant improvement in both makespan and reliability. Pakmehr et al. [26] introduced an energy-efficient and deadline-Aware task scheduling in fog computing approach, which anticipated the traffic of fog nodes, categorized them into low-traffic and high-traffic groups, and allocated jobs utilizing reinforcement learning and metaheuristic algorithms.

Yu and Zheng [27] developed a Hybrid Evolutionary Task Scheduling and Virtual Machine Placement method (HETSVP) for reliable fog computing, which combined PSO with virtual machine placement strategies, leading to decreased makespan and energy usage. Choppara and Mangalampalli [28] proposed a deep reinforcement learning-based task scheduler, DRLMOTS (Deep Reinforcement Learning based Multi Objective Task Scheduler in Cloud Fog Environment), for fog-cloud integration. It outperformed the existing algorithms in efficiency, fault tolerance, and energy consumption while improving task scheduling in fog-cloud environments.

3. Research method

An innovative structure for the delivery, use, and supply of computing services (including hardware, software, platforms, and other computing resources) via computer networks, such as the internet, is offered by cloud computing, a computational paradigm built on these networks. The continuous and growing interest in cloud computing and the desire to enhance its efficiency have led to the emergence of other cloud models, such as fog computing. Fog computing is considered an efficient and cost-effective model for executing processing tasks, and it has significant advantages over cloud computing architecture. However, achieving an efficient fog computing system and reaping its benefits is only possible if an optimized model for the performance of fog computing resources is established. This requirement can be met through resource allocation and optimal scheduling of task execution in the fog environment. Accordingly, this research presents a new method for the optimal allocation and scheduling of resources in fog computing systems. Before addressing the mechanisms of resource allocation and scheduling in the proposed method, it is essential to outline the system model and assumptions used in the design of the proposed method.

3.1. System model

The system model that was used to construct the suggested approach will be explained in this section. Based on the OpenFog consortium's standard design [29], which comprises of a cloud layer and a two-layer fog architecture, a cloud and fog system for computing is considered in this study. In addition, a set of isolated fog nodes is present for processing critical applications. All processing resources in this fog system have specified computing power, memory, and storage space. The assumed system model is depicted in Fig. 1.

According to Fig. 1, the assumed system model consists of the following components:

1. End Users or IoT Devices: This component includes the set of tasks created for execution in the fog layer, which belong to them.
2. User Applications: This component consists of two parts: 1) Workflow and 2) QoS requirements. The workflow includes a set of interdependent tasks, and the aim of the proposed method is to determine the location and order of execution for these tasks through the scheduling algorithm. The QoS requirements specify the execution needs for each set of tasks.
3. Fog Layer 1: This layer comprises fog resources, including a set of processing elements located in proximity to the end users. The priority of the proposed algorithm is to execute user tasks using resources available in this layer because executing tasks on these resources can lead to lower operational costs and shorter delays.
4. Fog Layer 2: This layer includes a set of processing resources that are located at a greater distance from the users. Accessing these computational resources will be more expensive, and a task will only be assigned to these resources if Fog Layer 1 is unable to provide the necessary resources for task execution.
5. Isolated Fog Resources: This layer includes a set of dedicated fog resources intended to meet specific execution scenarios (where some tasks require dedicated resources due to their QoS requirements). These resources are not shared and are solely allocated to the execution of a particular workflow.
6. Cloud Resources: An expensive set of processing resources that are of high computational power, used for the execution of tasks only when no fog layer computational resource is capable of performing the execution of the task.

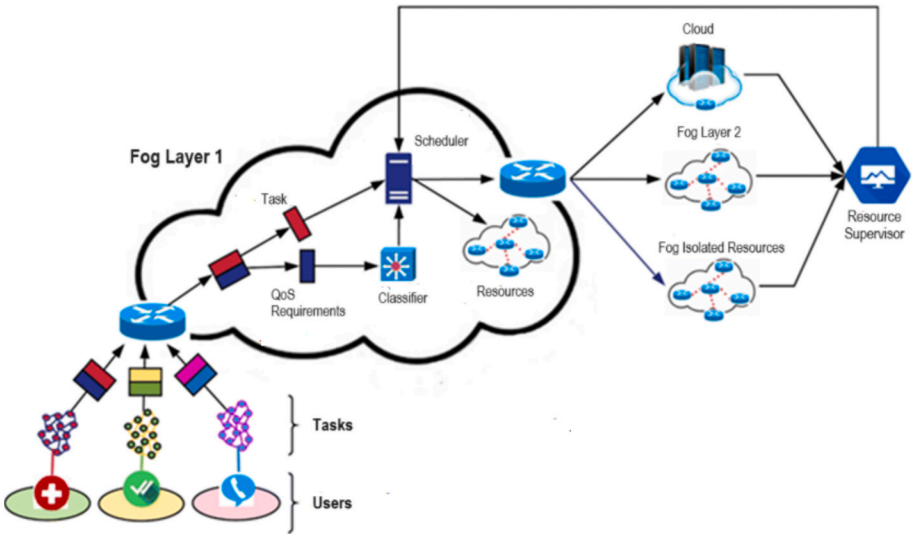

Fig. 1. System model assumed in the proposed method.

7. Classifier: The proposed system model uses a deep neural network-based classification model to identify the optimal processing resources for the execution of tasks. This classifier receives the characteristics of the task set and their QoS requirements as input, and then determines, through the output neuron weight vector, which resources are suitable for executing the input tasks. The output of this neural network is used as input for the scheduling algorithm.
8. Scheduling Component: This is the main part of the proposed system model, which uses the decisions made by the classification component (deep neural network) to determine the optimal computational resources in the fog layer. Once the optimal resource for executing each task is identified, the order of execution for each task on the processing resources is also determined using an improved version of the round-robin scheduling algorithm.
9. Resource Monitor: The proposed scheduling algorithm must continuously be aware of the status of the available computational resources. To this end, the status of resource utilization (processor and memory available for each resource) must be provided to the scheduling component in real-time. This task is the responsibility of the resource monitoring component, which facilitates the supply of the necessary information for the optimal scheduling of tasks.

According to the proposed system model, the end user sends an application to Fog Layer 1. Fog Layer 1 can be considered the edge of the computing network, consisting of a set of cost-effective resources. Each application comprises a workflow (including a set of interdependent tasks) and QoS requirements to be processed at the network edge. The QoS requirements represent the execution needs of the tasks, and the goal of the scheduling algorithm in the proposed method is to identify the computational resources that can meet these needs.

In the system model used in the proposed method, the QoS requirements are processed by a deep neural network-based classification model. This classification model categorizes the tasks according to their QoS requirements and determines the appropriate processing resource for executing each task. The result of this classification is used by the scheduling component. In the scheduling component, in addition to identifying the processing resource for executing the task, the order of execution for tasks on each resource is also determined using a round-robin scheduling algorithm. Finally, the input tasks and the scheduling list for their execution are sent to each computational resource. During the execution of the tasks, the status of the computational resources is monitored by the resource monitoring component, and the status changes of each resource are sent to the scheduling component. After stating the assumptions used in the system model, we will present the proposed method. The assumptions used in the proposed system model are as follows:

- Each computational resource in the fog layer has limited processing power and memory. Considering virtualization capabilities, the ability to execute tasks on each resource is independent of its hardware platform. Thus, each computational resource can simulate some of the necessary hardware for executing tasks using virtualization technology. However, the specifications of the virtualized system cannot exceed the processing capabilities of the resource.
- Given the capabilities of parallel processing, each fog computational resource can execute a limited number of computational tasks simultaneously. The amount of tasks that can be executed concurrently is relied on the amount of physical processor cores of the computational resource.
- The network components have varied communication properties because of the various technologies utilized to build the radio equipment in the Internet of Things structure and fog resources. The network is therefore regarded as diverse.

The continuation of this section is dedicated to explaining the introduced scheduling mechanism based on the system model and the above assumptions.

3.2. Proposed method

In this research, an IoT network consisting of several objects with limited processing power is considered, where each object has a set of tasks to process. Each object can perform tasks locally according to its limited processing power, which requires significant time and energy at the node. On the other hand, the aforementioned objects can offload their tasks to the computational resources available in the fog layer. This process helps conserve energy resources and reduces the latency caused by computations on user equipment. The goal of the presented model is to create a suitable communication framework between the nodes and the computational resources in the fog layers and the network edge, as well as to allocate and schedule the computational resources. The defined network structure in the introduced model is illustrated in Fig. 2.

According to Fig. 2, the defined network architecture includes a set of IoT objects with active communications, as well as a collection of processing resources at the edge, fog, and cloud layers. Peer-to-peer communications in the network are represented by solid lines, while communications with higher layers are depicted as dashed arrows. In the proposed method, the scheduling algorithm and the deep learning model identify the most suitable processing resources in the fog layer based on their data characteristics and the processing capacity of the computational resource. As a result of this process, the network's objects use less energy, have lower latency, and have better user experiences. Additionally, the supposed data will be forwarded to the upper layers for processing if it cannot be processed by the computing capabilities of the basic fog levels. Three stages make up the suggested approach in this study to accomplish this goal:

1. Establishment of Stable Communications: This step aims at establishing a stable communication infrastructure between the components of the network. In the proposed method, the minimum spanning tree structure based on node distance is used in determining the communications that are stable within the network.
2. Determining Fog Computational Resources for Each Task: As mentioned earlier, a deep neural network is used in the proposed method to perform this step. This neural network receives the characteristics of the task and the computational resources as input and determines the priority of task allocation to each resource in the form of a weighting vector.
3. Scheduling Task Execution on Each Resource: After assigning each task to one of the available resources, the scheduling component in the proposed architecture will independently determine the order of task execution for each resource. To this end, an improved version of the round-robin algorithm will be employed, wherein the sequence of task execution and the processing time for each task are determined based on the dependencies between the tasks.

The steps of the proposed method are shown as a flowchart in Fig. 3. The next step is to describe these phases in the proposed method.

3.2.1. Establishing stable communications

In order to identify reliable links among things and computational assets at the fog and network edge, the suggested method's first step is to build a tree-like arrangement for communication inside the network. Hello control packets are exchanged to facilitate this activity, which starts with determining a list of neighbors. By including its own unique identification in the body of a control packet and subsequently broadcasting, each node uses this method to promote itself to its neighbors. Each node sends its stored sender identifier in its neighbor list upon reception. More important, each node stores the strength of the signal which is received by the neighbor node in its memory. Since this process is to be repeated from every node in the network, each node will have a list containing neighbor information and also the strength of the signal

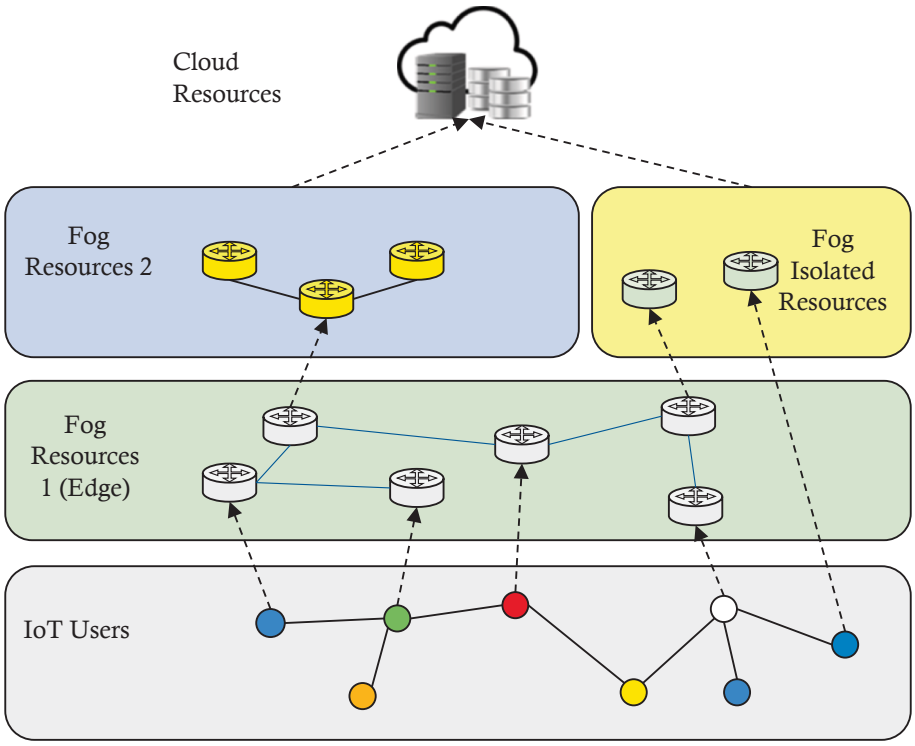

Fig. 2. The defined network structure in the introduced model.

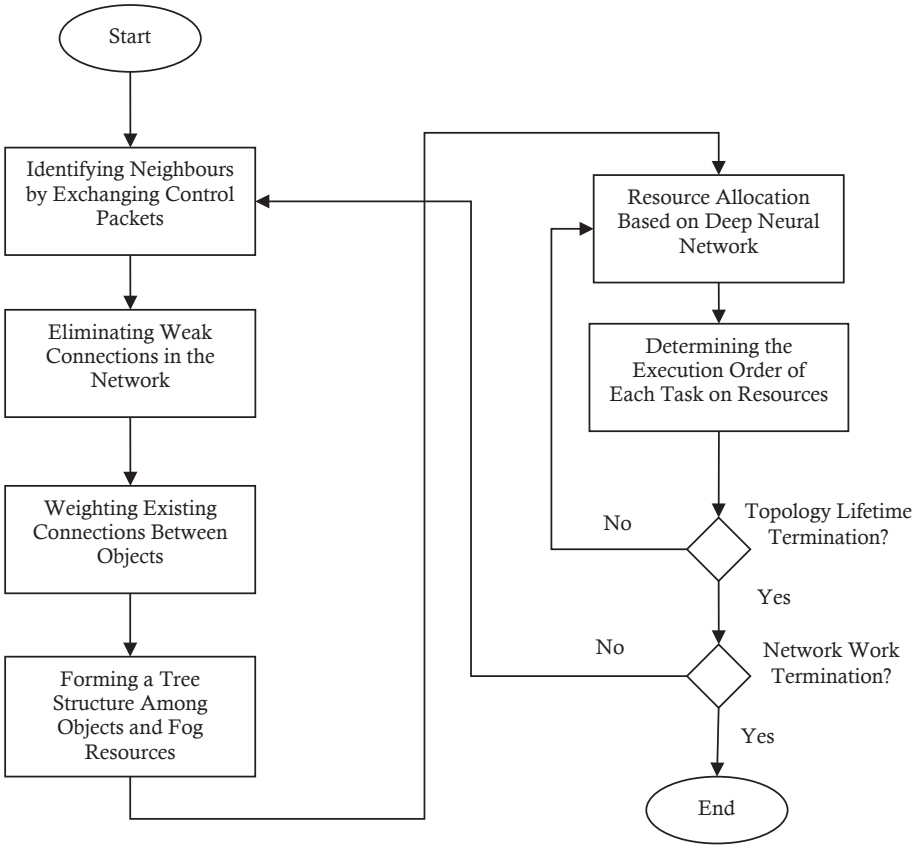

Fig. 3. The introduced approach as a flowchart.

of all neighbors.

The next step involves exchanging the neighbor lists to eliminate low-quality connections in the network. Each node relays to its nearby nodes the strength of the signal it receives in this stage. Assume that node B sends a signal with a strength of $RSSI_{A,B}$ to node A. Nodes A and B will know how strong of the signal they received from one another after exchanging signal intensities. Each node in this scenario, like A, will assess the connection's quality with node B in the manner described below:

- With the received signal strengths from both ends of the connection, node A calculates the average signal strength received by itself and its neighbor. The purpose of this is to reduce the detrimental effects of noise in the communications.
- If the average signal strength is greater than a threshold P, the connection between nodes A and B is considered active; otherwise, this connection will be removed from the list of active connections in the network due to insufficient quality. The stability of a link depends a lot on the threshold P for average RSS. The value of P was found by performing a series of initial experiments. The purpose of these experiments was to find the best way to maintain a strong network and dependable connections. Based on what was seen in the first evaluations, a threshold value of −75 dBm was chosen. Those connections with an average RSS below P were considered unstable and therefore not included in the set of active connections used to build the communication tree.
- Each node is going to remove the other from its neighbor lists if the link connecting nodes A and B is of insufficient quality.

After this process is completed by all nodes in the network, a set of bidirectional connections with sufficient quality will be established between the wireless network nodes. Following this, the primary connections of the network will be organized into a tree structure. To achieve this, we employ a method for weighting connections and forming a maximum spanning tree among the objects (users) as well as the fog and edge computational resources. Every active device communicates with its nearby nodes during this phase by sending a tree creation packet containing its ID, wireless range, and position data. The information gathered is then sent to the device having the greatest number of neighbors by each surrounding node. Once all the tree-building packets are routed to the node with the most neighbors—henceforth called as central node—this process is continued. The central node generates a view of the network nodes' communication structure after collecting these control messages from every node that is actively participating in the network. Based on this data, it can then generate an adjacency matrix of the nodes' connections.

The central node will determine the order of the active links among the devices once the adjacency matrix has been created. This node uses a distance measure among the devices to weight the adjacency graph's links. In order to accomplish this, the following formula is employed to determine the value of the link among nodes i and j:

$$W(i,j) = \frac{R_i + R_j}{\boldsymbol{D_{ij}}} \tag{1}$$

In the above relationship, the metric R_i represents the radio range for device i. Additionally, $D_{i,j}$ denotes the Euclidean distance among the devices, which is measured based on samples of the received signal strength. The tree construction procedure in the suggested method makes use of the value W(i,j), which in Equation (1) denotes the importance of the link among i and j, for building the spanning tree. Prim's algorithm will be used by the central node to build the maximum spanning tree in order to achieve this. According to this technique, the network's spanning tree arrangement consists of the most useful links currently present. The links with the greatest weights are utilized to create this collection of connections, and the selection process is carried out so that the tree structure is free of cycles. After constructing the spanning tree according to the aforementioned process, the allocation of fog computational resources to the tasks will take place. We will elaborate on this step next.

3.2.2. Determining fog computational resources for each task

In the proposed approach, a deep neural network computes the optimal fog computational resources that shall execute the input tasks. It takes the characteristics of tasks and computational resources as inputs and decides on the priority of task allocation to each resource in the form of a weighting vector. The proposed method employs a CNN. Fig. 4 illustrates the architecture of the deep neural network used in the proposed methodology.

As shown in Fig. 4, the proposed CNN for determining the processing resources to execute each input task is made up of the following components:

An Input Layer: This layer serves as the first layer in the proposed CNN, responsible for receiving the characteristics of the tasks and the available computational resources, and transferring these features to the two-dimensional convolution layer. The set of features applied to the input layer of the deep neural network includes:

1. The execution time of the task and the processing and memory capacity required for its execution.
2. The input dependency of the task on other tasks, described in the form of a vector like $D = \{d_1, \cdots, d_n\}$ (where n indicates the total number of tasks). In this vector, if the input of the current task is dependent on the output of task i, then $d_i = 1$; otherwise, that entry will be zero.
3. The estimated computational energy required for executing the current task, which is assessed using the model presented in [30].
4. The list of unused processor and memory capacity in each computational resource, described in sets $C = \{c_1, \cdots, c_R\}$ and $M = \{m_1, \cdots, m_R\}$ respectively (where R denotes the number of computational resources).

The above set of features is combined into a vector format to furnish the required input to the neural network.

Two 1D Convolutional Layers: The convolutional layers used in the proposed CNN model will be one-dimensional since the input data is in vector form. In the first convolutional layer, the width of the filter is set to 5, while for the second convolutional layer it is set to 3, and the number of filters in them is 16 and 32, respectively. It is noteworthy that these values for the dimensions and the number of filters in the convolutional layers were determined empirically using a grid search strategy. Based on the analysis of the experimental results, using more than 16 and 32 filters in these layers led to overfitting. The reason for employing two convolutional layers in the proposed CNN is to achieve different levels of abstraction in the input features. In this model, the first convolutional layer describes the detailed features of the data, while the second convolutional layer (which receives the sampled features from the first Max Pool layer as input) extracts deeper features from the data. Thus, in each of these convolutional layers, the input characteristics are processed using the filters available in that layer, creating various patterns to describe the task features. The use of this number of filters enables the development of a more powerful neural network for efficiently describing the features.

Two ReLU Layers and Batch Normalization (BN): Each pair of ReLU and BN layers in the proposed CNN model is applied directly after the convolutional layers. The batch normalization layer in CNN transforms the output distribution of the convolutional layer into a standard normal distribution by being placed between the convolutional layer and the ReLU activation layer. This process will enhance training stability, accelerate the learning process, and increase model accuracy while reducing dependence on the initial parameter selection of the network. As it stabilizes the input distribution of the activation layer, the

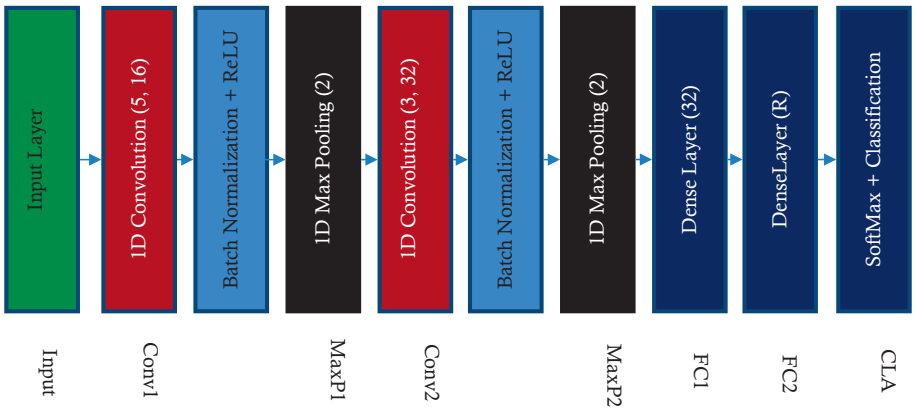

Fig. 4. The structure of the deep neural network used in the proposed method to determine processing resources in the fog layer.

gradients become more stable to avoid problems of vanishing or exploding gradients. Moreover, this layer will allow the model to optimally adjust the data distribution using the learnable scale and shift parameters for better performance. Conversely, the ReLU layers act as an activation function to transform the obtained data from the convolutional layer. The ReLU function is used in the proposed CNN model because of the nature of the input to this neural network, since the data for the input to describe resource and task characteristics are positive numbers. Hence, functions like a sigmoid will leak the irrelevant features to other layers. In other words, this function can be said to be-for any negative output from convolutional, the value is kept to zero; otherwise, it is directly passed to the next layer.

Two 1D MaxPool Layers: The goal of these layers is to reduce the spatial size of the features obtained through the convolutional layers. The window size in these layers is set to 2, thereby halving the dimensions of the features extracted from the convolutional layers.

Fully Connected Layer: This layer is used to convert the features obtained from the previous layers into vector form. The first fully connected layer used at the end of the proposed CNN has 64 neurons. This layer describes the relationships between the features of the resources and the input task in the form of a vector of length 64. In contrast, the second fully connected layer has a size R, which corresponds to the number of available computational resources, and translates the mapped features into posterior probabilities indicating the likelihood of the sample belonging to each of the target classes.

Classification Layer: The classification layer is a layer responsible for the output of the CNN. The number of neurons in this layer is equal to the number of available resources. Therefore, the output weight of each neuron represents the optimality of a corresponding computational resource for the execution of the task.

Based on the described structure for the proposed deep neural network, the characteristics of the tasks and the computational resources are applied to the input layer of this network, and the neural network model will determine the optimal processing resource for executing the task based on the weight values obtained from its classification layer. In this case, the input task will be suitable for execution on the resource for which the output neuron weight corresponding to that resource has the highest value.

It is important to note that the training of the deep neural network is performed through an evaluation phase in the proposed method. For this purpose, during the evaluation phase, after each determination of the processing resource by the neural network, the waiting time obtained is compared with the average waiting time from previous periods. If the waiting time of the tasks in the current step is less than the average from previous steps, the performance of the deep neural network in the recent step is considered correct and is recorded as a training data point with a positive label. Otherwise, the weights determined by the neural network in the recent period are deemed incorrect, and this data is removed from the training set. By continuously repeating this process, a set of training data for the deep neural network will be formed, which will enable minimizing the waiting time for tasks.

3.2.3. *Scheduling task execution on each resource*

In the proposed method, after assigning each task to one of the available resources, the scheduling of task execution on each computational resource is performed independently. This process is carried out using the scheduling component in the proposed architecture. To this end, the list of tasks assigned by the deep neural network to each resource, denoted as r, is organized in the form of a list like $A_r = \{t_1^r, \cdots, t_n^r\}$. In this set, t_i^r represents the i-th task assigned to the computational resource r in the fog layer. This task is described based on the remaining execution time and its input requirements. A task can only be executed if its execution requirements are met (i.e., the necessary input data for the task is available). Therefore, the proposed scheduling algorithm aims to prioritize tasks whose outputs are needed for the execution of other tasks. This prioritization is implemented in the proposed scheduling algorithm using an improved version of the round-robin algorithm, with the difference being that the order of task execution and the processing time for each task are determined based on the dependencies among the tasks. Given these explanations, the proposed scheduling algorithm for determining the order of task execution on each processing resource, such as r, includes the following steps:

Step 1: The execution list T is initialized as $T = \varnothing$.

Step 2: The list of tasks assigned to the current resource is stored in the order of arrival as $A_r = \{t_1^r, \cdots, t_n^r\}$.

Step 3: Each task is assigned an initial execution priority with a value of zero, $P_r = \{0, \cdots, 0\}$.

Step 4: For each task t_i^r, the input requirements of the task are extracted. One of the following conditions will be met:

- If all the input requirements of the task t_i^r are available, it is added to the execution list T, $(T = T \cup t_i^r)$.
- Otherwise, the list of tasks whose outputs are part of the input requirements t_i^r is extracted, and the execution priority of each of them is increased by one $P_r = P_r + \delta_i^r$. In this relation, δ_i^r is a binary function that has a value of one for each task that t_i^r depends on, and otherwise, this function will have a value of zero.

Step 5: The execution list T is sorted in descending order based on the execution priority values P_r.

Step 6: Each task in the execution list T is executed for the duration of the processor's quantum time, and completed tasks are removed from the sets T, A_r, and P_r.

Step 7: If $A_r = \varnothing$, the algorithm terminates; otherwise, it repeats from Step 1.

4. Research finding

In this section, a simulated network is implemented using MATLAB 2020a software and an attempt is made to examine the network performance in this simulation environment. In the simulations, 100 IoT users are considered, each of which can generate one of three types of tasks: processing, storage, or a combination of the two. These tasks can be executed by fog resources, and the number of fog resources in the network is considered to be 15, so that these 15 resources will be responsible for executing the tasks of the IoT nodes. The experiments were repeated 30 times and the presented results in this section are the average values. Table 1 shows the simulation parameters.

To comprehensively evaluate the performance of the proposed task scheduling method, we designed two distinct experimental scenarios:

- Varying Number of Tasks: Designed to check the system's ability to function well when the workload changes. The number of tasks that can be executed at the same time is set to vary from 50 to 250, to show how the fog resources are used at different demand levels. It helps us see how the proposed method and other algorithms react to more work being done on the system.
- Variable Average Task Completion Time: Examines how the system does when handling tasks that require different amounts of computational effort. Here, the time needed to execute tasks is changed in a controlled way, from light to heavy, without greatly changing the number of tasks. It allows us to see how scheduling approaches work with a range of task challenges and the effects this has on resource use and system performance.

Both scenarios are compared to three baseline models from recent papers which are chosen to represent a variety of important scheduling approaches. Since the Enhanced Round-Robin Heuristic Algorithm [19] brings an advanced heuristic method, we can compare it to our own improved Round-Robin component. We compare our method with a PSO-based technique, as these are common in scheduling problems [20]. To compare our work with recent AI-based solutions, we also discuss DRLMOTS [28] which uses a different approach (DRL) for task scheduling in fog computing. All these methods were put into the simulation environment and tested using the same scenarios and data to ensure fairness. With this selection, we can test our CNN-based method against well-known heuristic, metaheuristic and learning-based strategies using response time, turnaround time and waiting time as the main metrics.

Fig. 5 shows the error changes of the deep neural network during the training phase. The neural network is implemented in such a way that the Adam optimizer is used to train it and the min batch size is set to 32. The time required to collect training data is set to 1000 s of the simulation; thus, the data required for training the neural network is generated based on the first 1000 s of network node operation. The output values obtained for the loss function in different iterations are shown in Fig. 5, which shows that with progress in different cycles, the loss rate has significantly decreased and has reached close to zero. This loss reduction is based on the training data and the loss rate for the test data is also close to the training data, which indicates data alignment and proper functioning of the neural network in the training phase. Examination of the training data shows that there are no problems such as overfitting or underfitting, and the pattern of loss changes in different cycles for the training and test data is almost consistent.

Table 1
Simulation parameters.

Parameter	Value
Number of Tasks per Resource	100–250
Task Arrival Time	0–500 ms
Task Execution Time	50–200 ms
Deadline Rate	10 %
Quantum Range	1 μs–10 ms
Number of Fog Resources	15

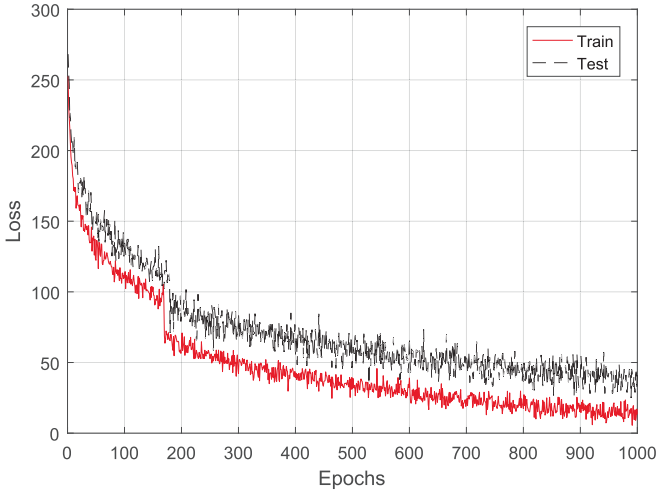

Fig. 5. Graph of the deep neural network error changes during the training phase.

4.1. Changes in the number of tasks

In this scenario, the number of tasks ready to be executed at any given time was set variably, between 50 and 250 tasks. The purpose of this change in the number of tasks was to examine the system's performance under different conditions and evaluate the efficiency of the scheduling algorithm. Using the scheduling algorithm, an attempt was made to determine the optimal execution of these tasks for the system. This process was designed in such a way that it was possible to analyze the impact of the number of tasks on system efficiency and response time, and ultimately to achieve results appropriate to different conditions.

Fig. 6 shows the average response time for changes in the number of tasks, which indicates that as the number of tasks increases, the average response time also increases. Response time refers to the time interval during which the selected source node processor first executes parts of the computational process. More precisely, response time refers to the average time from the time the task is sent to the time the first response is received from the source node processor. This graph shows that with

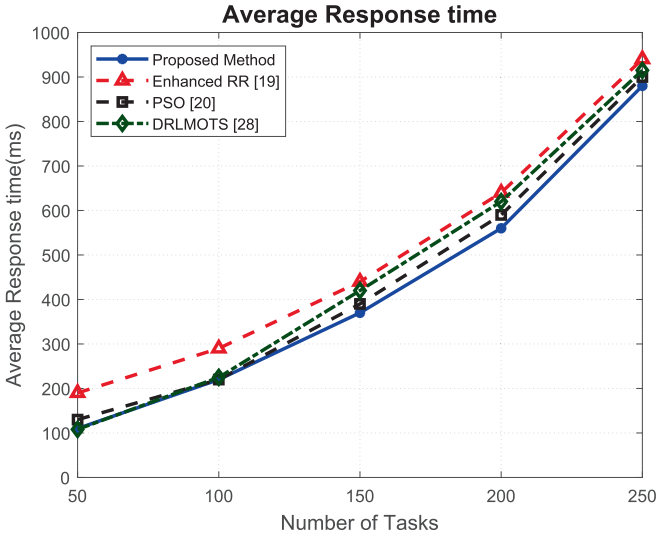

Fig. 6. Average response time for the number of tasks.

the increase in the number of tasks, the traffic load of the network and the computational system increases, and as a result, the response time also increases. This upward trend of the graph is due to the increase in the computational load of the entire system. Based on the results obtained, the proposed method has been able to provide less waiting time than the enhanced RR [19] method and the PSO-based [20] method compared to the compared methods. This is because the deep learning model combined with the improved RR-based model brings such an outcome. Unlike the PSO method, our model adopts a spatial search strategy and analyzes data associated with the computation patterns of every source node in order to find the proper source node by its output. Thus, this approach results in the reduction of time spent for the search of an appropriate source, compared to optimization-based methods.

Fig. 7 shows the turnaround time, which is a measure of the time elapsed from the moment a task is submitted to its completion. This time interval is known as the turnaround time, and the turnaround time increases as the computational load on the system increases. However, in different iterations and conditions, the proposed method was able to record a shorter time in terms of the turnaround time criterion. The reason for this improvement is the use of resources that can execute tasks in a shorter time interval. These results show that the CNN has performed better than the enhanced RR [19] method and the PSO-based [20] method in selecting resources that execute tasks correctly. In addition, the use of the round-robin strategy in the proposed method also had a positive effect on reducing the turnaround time. The results show that the proposed method has a good performance in preparing the final response for executing a task, and these two measures clearly indicate that the proposed method has performed effectively in responding to the execution of tasks.

Fig. 8 shows the average waiting time. This is the time a task waits in the execution queue to be executed by the processor. In general, waiting time refers to the total sum of the time periods that a task waits in the execution queue. In this RR-based method, tasks are executed at different time intervals, and the times that a task remains in the queue are also considered as interruptions as part of the waiting time. The results obtained show that the proposed method has been able to achieve a significant reduction in waiting time in most time intervals compared to the enhanced RR [19] and PSO [20] methods. The reason for this improvement is the use of the improved RR mechanism in the proposed method. By combining the results of the decision-making of the deep neural network and involving these decisions in the execution of the tasks of the RR algorithm, this method prioritizes the execution of tasks based on the waiting time, and as a result, it has made it possible to

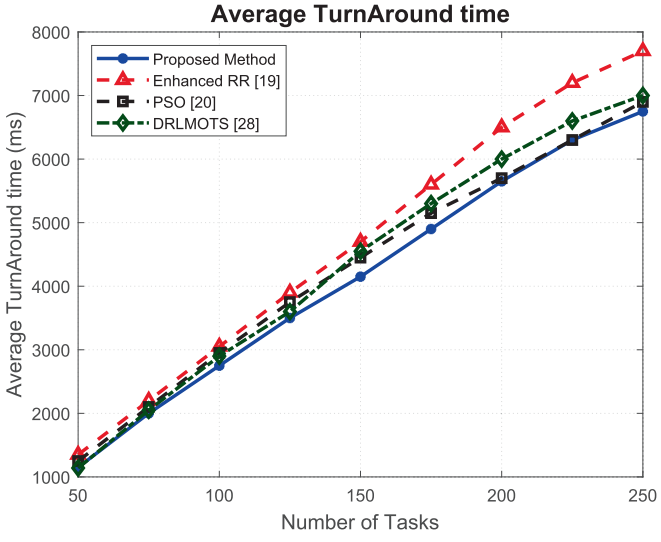

Fig. 7. Average time a task spends in the scheduling queue for the number of tasks.

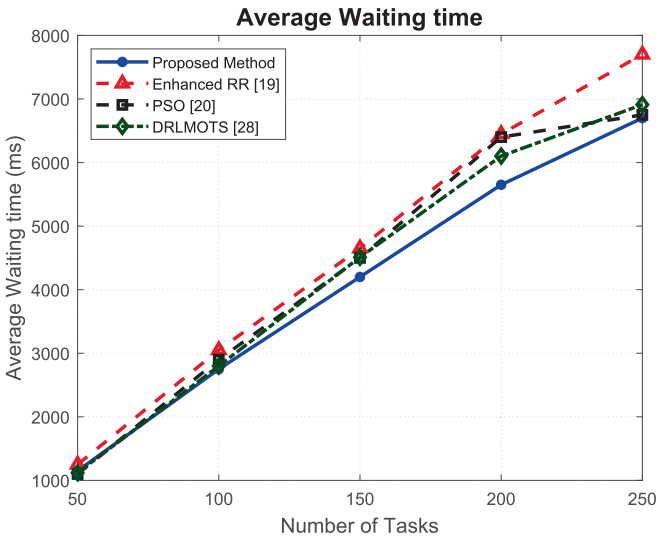

Fig. 8. Average waiting time for the number of tasks.

reduce the waiting time. The order of task execution based on waiting time is one of the priorities of the proposed algorithm, which confirms the correct performance of this method in this context.

4.2. Changing the average task completion time

In this scenario, the task completion time is examined and the average time required to execute each task is changed. The situation is changed from light tasks to heavy tasks to better characterize the different effects of computational load on system performance. Three main time metrics are considered to evaluate the proposed method and other methods. This evaluation helps in a more detailed analysis of the efficiency and effectiveness of each method and shows that changing the type and amount of task load can have significant effects on the completion time and system efficiency.

Fig. 9 shows the average response time versus task completion time changes, illustrating the significant improvement of the proposed method over the compared methods. This difference is clearly evident, and one of the factors that has been effective in reducing the response time of the proposed method is the use of the proposed structure to determine stable network connections. In this way, they were able to

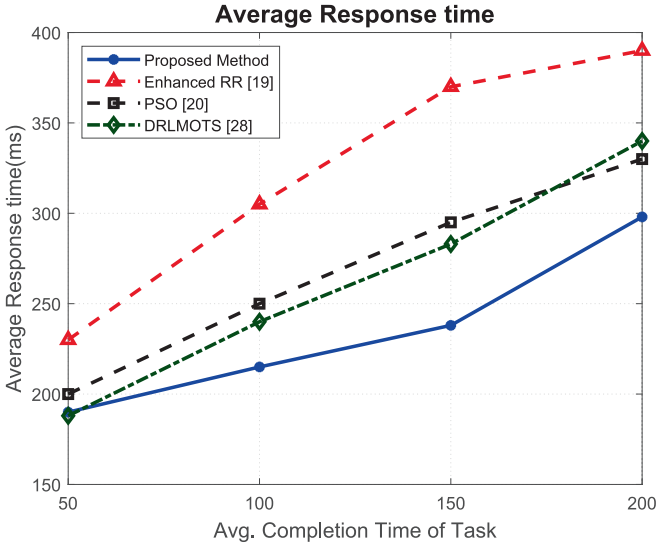

Fig. 9. Average response time for input processing times.

bring forward those relationships which, by offloading tasks to computational resources, minimized time and again reduced response time. The proposed CNN model for finding the optimal computational resources has also been very effective in response time reduction.

Figs. 10 and 11 show the average turnaround and wait times. In these graphs, which are seen in ascending order, it is clear that as the task completion time increases, the turnaround time and waiting time also increase. This trend seems quite logical, because as the task completion time increases, the tasks remain in the execution queue for a longer period of time and, as a result, more time is required to process each task. This process clearly affects the increase in the turnaround time and waiting time and causes the overall efficiency of the system to decrease. The proposed method is able to help the optimization process through the communication structure used and the use of a coherent approach. Combining this structure with the RR algorithm to determine the order of task execution can have a positive effect on improving efficiency. Also, the use of a deep neural network to determine the optimal resources in executing tasks provides more flexibility in responding to processing needs. These methods have been able, in sum, to reduce the turnaround time and waiting time compared to the compared methods, hence improving the performance of the overall system. This result shows the advantages of the proposed method over other methods in the field of optimization and enhancement of efficiency.

Table 2 shows the experimental results, the time differences between the proposed method, the improved Round Robin algorithm and the PSO-based scheduling in terms of response times, turnaround time and waiting time.

In the case with a variable number of tasks, the proposed method with an average response time of 425.1 ms shows an improvement of 15.13 % compared to the Enhanced RR [19] algorithm and 4.79 % compared to PSO-based scheduling [20]. The turnaround time of the proposed method is 4089 ms, which represents an improvement of 12.16 % compared to Enhanced RR [19] and 3.45 % compared to PSO-based scheduling [20], while the waiting time of 4039 ms shows improvements of 12.27 % and 5.12 % over Enhanced RR [19] and PSO-based scheduling [20], respectively. In the case with a variable completion time, the response time of the proposed method reaches 233.6 ms, showing improvements of 27.54 % over Enhanced RR [19] and 12.97 % over PSO-based scheduling [20]. The turnaround time is reduced to 3531 ms, an improvement of 10.19 % over Enhanced RR [19] and 7.69 % over PSO-based scheduling [20], and the waiting time is reduced to 3468 ms, representing improvements of 10.36 % and 4.30 % over Enhanced RR [19] and PSO-based scheduling [20], respectively.

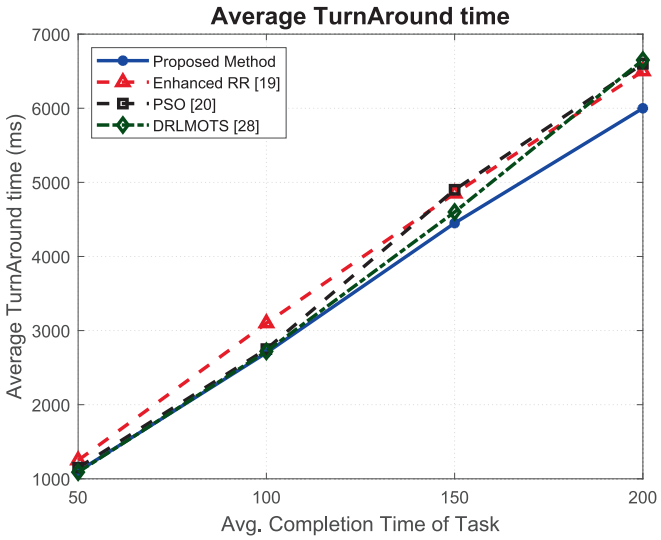

Fig. 10. Average time a task spends in the scheduling queue for input processing times.

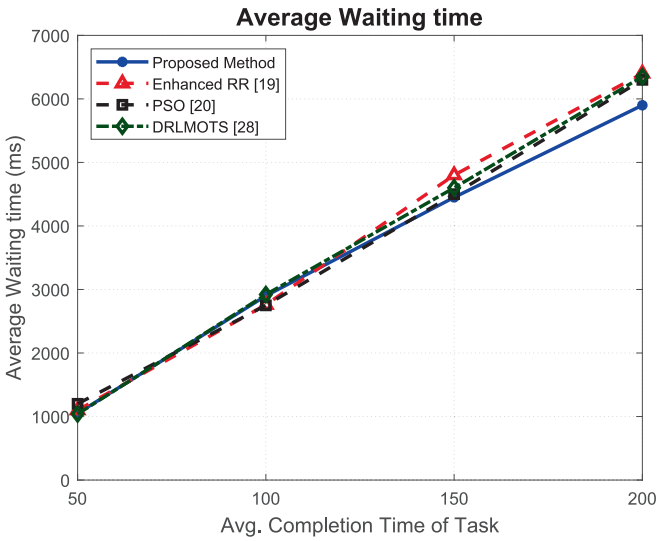

Fig. 11. Average waiting time for input processing times.

These results indicate that the proposed method performs significantly better than the other two methods.

5. Conclusion

Due to the rapid growth of the IoT Things and the increase in data volume, the need for efficient and scalable computing systems is strongly felt. In this paper, an innovative deep learning-based method for effective task allocation and scheduling in fog computing systems is proposed. The experimental results show significant improvements in response times, turnaround times, and waiting times in different scenarios. So that this method has achieved improvements of 9.96 %, 7.80 %, and 8.69 % in the first scenario and 20.25 %, 8.94 %, and 7.33 % in the second scenario. Results showed that very high effectiveness under different work conditions could be reached, and system performance can be hugely improved with the proposed method for optimizing the timing. As a result, the proposed method can be considered as an effective solution for optimizing resource allocation and task scheduling in fog computing systems and help build more robust and efficient infrastructures to support IoT applications. Listed below are the current limitations and potential areas for future research:

i. The first limitation is related to the limited number of scenarios that the proposed method is evaluated in this evaluation and is focused on two specific scenarios of the computing system. This means that the results may not be generalizable to other scenarios and therefore, ensuring the generality of the performance of this model in different conditions requires evaluation in more diverse scenarios.
ii. The second limitation refers to the simplified assumptions of this model, such that some of the assumptions considered in this system are, in fact, simplified examples of real-world realities that may not be very true in the accurate modeling of this system in real conditions. Assuming the processing capacity of the resources as well as the connections between the nodes requires considering more detailed models to prepare this model for implementation in real conditions. In the future, efforts will be made to generalize and improve the proposals in these two aspects.

CRediT authorship contribution statement

ZhongYI Huang: Investigation.

Table 2
Summary of the results of the experiments.

Experiment	Measure	Proposed	Enhanced RR [19]	PSO-based [20]	DRLMOTS [28]
Variable No. Of Tasks	Response Time	425.1 ms	500.9 ms	446.5 ms	457.6 ms
	Turnaround Time	4089 ms	4655 ms	4235 ms	4348 ms
	Waiting Time	4039 ms	4604 ms	4257 ms	4288 ms
Variable Completion Time	Response Time	233.6 ms	322.4 ms	268.4 ms	268.3 ms
	Turnaround Time	3531 ms	3932 ms	3825 ms	3765 ms
	Waiting Time	3468 ms	3869 ms	3624 ms	3727 ms

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Klein S. IoT solutions in microsoft's azure IoT suite. Berkeley, CA: Apress; 2017. p. 273–89.
[2] Laroui M, Nour B, Moungla H, Cherif MA, Afifi H, Guizani M. Edge and fog computing for IoT: a survey on current research activities & future directions. Comp Commun 2021;180:210–31.
[3] Bonomi F, Milito R, Zhu J, Addepalli S. August). fog computing and its role in the internet of things. In: In Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing; 2012. p. 13–6.
[4] Hosseinzadeh M, Azhir E, Lansky J, Mildeova S, Ahmed OH, Malik MH, et al. Task scheduling mechanisms for fog computing: a systematic survey. IEEE Access 2023; 11:50994–1017.
[5] Kotb Y, Al Ridhawi I, Aloqaily M, Baker T, Jararweh Y, Tawfik H. Cloud-based multi-agent cooperation for IoT devices using workflow-nets. J Grid Comp 2019;17 (4):625–50.
[6] Ren Z, Lu T, Wang X, Guo W, Liu G, Chang S. Resource scheduling for delay-sensitive application in three-layer fog-to-cloud architecture. Peer-to-Peer Networking Appl 2020;13:1474–85.
[7] Guevara JC, Da Fonseca NL. Task scheduling in cloud-fog computing systems. Peer-to-Peer Networking Appl 2021;14(2):962–77.
[8] Zhang M, Zhou Y, Quan W, Zhu J, Zheng R, Wu Q. Online learning for IoT optimization: a Frank–Wolfe adam-based algorithm. IEEE Internet Things J 2020;7 (9):8228–37.
[9] Batista DM, da Fonseca NL, Miyazawa FK, Granelli F. Self-adjustment of resource allocation for grid applications. Comp Networks 2008;52(9):1762–81.
[10] Bittencourt LF, Goldman A, Madeira ER, da Fonseca NL, Sakellariou R. Scheduling in distributed systems: a cloud computing perspective. Comp Sci Rev 2018;30: 31–54.
[11] Zhou Z, Wang H, Shao H, Dong L, Yu J. A high-performance scheduling algorithm using greedy strategy toward quality of service in the cloud environments. Peer-to-Peer Networking Appl 2020;13(6):2214–23.
[12] Najafizadeh A, Salajegheh A, Rahmani AM, Sahafi A. Multi-objective Task Scheduling in cloud-fog computing using goal programming approach. Cluster Comp 2022;25(1):141–65.
[13] Hosseini E, Nickray M, Ghanbari S. Optimized task scheduling for cost-latency trade-off in mobile fog computing using fuzzy analytical hierarchy process. Comp Networks 2022;206:108752.
[14] Yadav AM, Tripathi KN, Sharma SC. A bi-objective task scheduling approach in fog computing using hybrid fireworks algorithm. The J Supercomputing 2022;78(3): 4236–60.
[15] Navaneetha Krishnan M, Thiyagarajan R. Multi-objective task scheduling in fog computing using improved gaining sharing knowledge based algorithm. Concurrency and Comp: Practice and Experience 2022;34(24):e7227.
[16] Azizi S, Shojafar M, Abawajy J, Buyya R. Deadline-aware and energy-efficient IoT task scheduling in fog computing systems: a semi-greedy approach. J Network and Comp Appl 2022;201:103333.
[17] Ghafari R, Mansouri N. An efficient task scheduling in fog computing using improved artificial hummingbird algorithm. Journal of Comp Sci 2023;74:102152.
[18] Saif FA, Latip R, Hanapi ZM, Shafinah K. Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing. IEEE Access 2023;11: 20635–46.
[19] Alhaidari F, Balharith TZ. Enhanced round-robin algorithm in the cloud computing environment for optimal task scheduling. Computers 2021;10(5):63.
[20] Alsaidy SA, Abbood AD, Sahib MA. Heuristic initialization of PSO task scheduling algorithm in cloud computing. J King Saud Univer-Comp Inform Sci 2022;34(6): 2370–82.
[21] Iftikhar S, Ahmad MMM, Tuli S, Chowdhury D, Xu M, Gill SS, et al. HunterPlus: AI based energy-efficient task scheduling for cloud–fog computing environments. Internet Things 2023;21:100667.
[22] Abdel-Basset M, Mohamed R, Sallam KM, Hezam IM. Multi-objective task scheduling method for cyber–physical–social systems in fog computing. Knowl-Based Syst 2023;280:111009.
[23] Choppara P, Mangalampalli S. Reliability and trust aware task scheduler for cloud-fog computing using advantage actor critic (A2C) algorithm. IEEE Access 2024.
[24] Hussain M, Nabi S, Hussain M. RAPTS: resource aware prioritized task scheduling technique in heterogeneous fog computing environment. Clust Comput 2024;27 (9):13353–77.
[25] Hosseini SM, Shirvani MH, Motameni H. Multi-objective discrete cuckoo search algorithm for optimization of bag-of-tasks scheduling in fog computing environment. Comput Electr Eng 2024;119:109480.
[26] Pakmehr A, Gholipour M, Zeinali E. ETFC: Energy-efficient and deadline-aware task scheduling in fog computing. Sustain Comp: Inform Sys 2024;43:100988.
[27] Yu D, Zheng W. A hybrid evolutionary algorithm to improve task scheduling and load balancing in fog computing. Clust Comput 2025;28(1):1–26.
[28] Choppara P, Mangalampalli S. An efficient deep reinforcement learning based task scheduler in cloud-fog environment. Clust Comput 2025;28(1):1–26.
[29] Openfog Consortium. (2017). "OpenFog Reference Architecture for Fog Computing", Available online at: https://www.iiconsortium.org/pdf/OpenFog_Reference_Architecture_{20}9_17.pdf (Accessed on: June 08, 2024).
[30] Ebadifard F, Babamir SM. A PSO-based task scheduling algorithm improved using a load-balancing technique for the cloud computing environment. Concurrency Comput Pract Exper 2018;30(12):e4368.